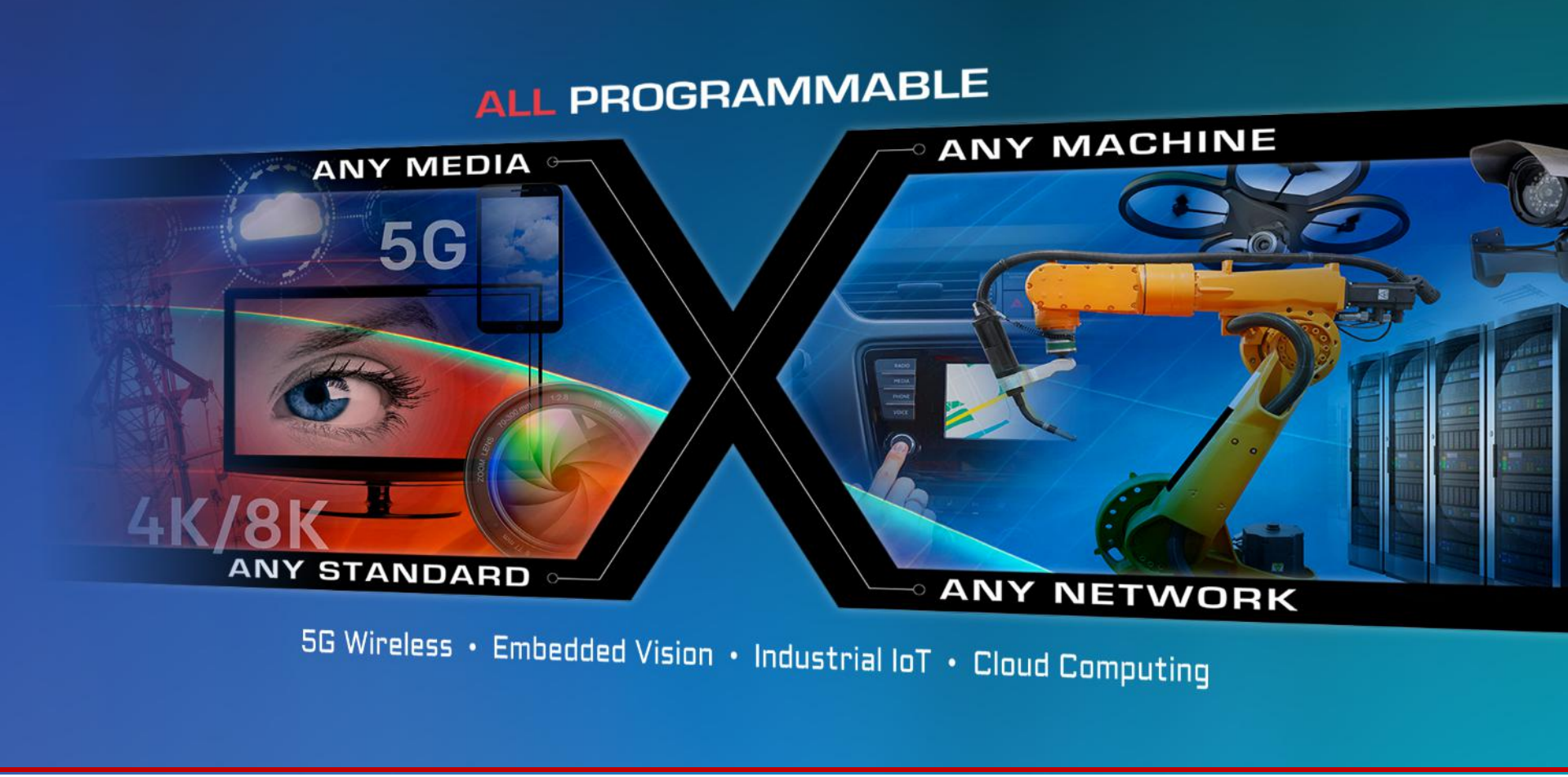


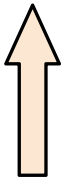
XILINX
ALL PROGRAMMABLE™

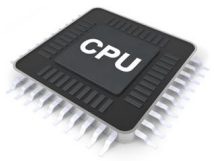
PYNQ™ Python Productivity for Zynq

Timeline

- **Session 1**
  - First steps with PYNQ-Z2
  - Labs:
    - Getting started with Jupyter Notebooks
    - Getting started with Ipython
    - Exploring PYNQ-Z2
    - Programming on-board peripherals
- **Session 2**
  - Overlay concept
  - Base Overlay
  - Labs:
    - Grove temperature sensor
    - Grove light sensor
    - Grove ledbar
    - OLED
- **Session 3**
  - Overlay design methodology
  - Building customized overlay
  - Labs:
    - Developing a single IP
    - Creating a python-driver
    - Creating a custom overlay
- **Session 4**
  - Why use Zynq for DL Inference
  - Binary neural network (BNN)
  - Labs:
    - MNIST + Char Recognition
    - Cifar 10
    - SVHN
    - Road Signs

The rise of productivity languages



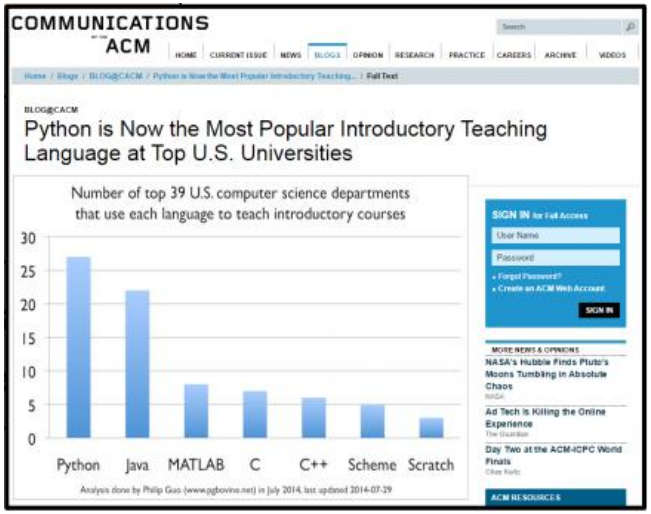


Applications,
Programming Frameworks

Productivity Languages,
Python, Scala, ..

Systems/Efficiency Languages
C, C++, OpenCL

Hardware Systems
OS, hypervisor, drivers


COMMUNICATIONS of the ACM

BLOG@CACM

Python is Now the Most Popular Introductory Teaching Language at Top U.S. Universities

Number of top 39 U.S. computer science departments that use each language to teach introductory courses

30
25
20
15
10
5
0

Python Java MATLAB C C++ Scheme Scratch

Analysis done by Philip Guo (www.pgbovine.net) in July 2014, last updated 2014-07-29

SIGN IN for Full Access

User Name

Password

SIGN IN

MORE NEWS & OPINIONS

NASA's Hubble Finds Pluto's Moons Tumbling in Absolute Chaos

Ad Tech Is Killing the Online Experience

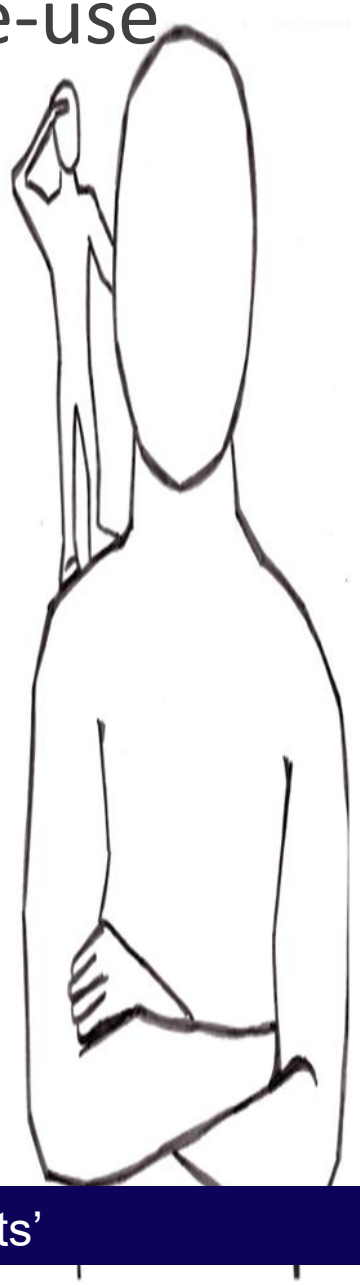
Day Two at the ACM-ICPC World Finals

ACM RESOURCES

How can we program ZYNQ in a productivity language?

The rise of open source and knowledge re-use

- **GitHub: world's leading software repository**
  - https://github.com/
- **Stack Overflow: communities helping each other**
  - http://stackoverflow.com/
- **Python Package Index (PyPI): 80,000+ packages**
  - https://pypi.python.org
- **Jupyter Notebooks: literate, reproducible computing**
  - http://jupyter.org/
- **Readthedocs: software documentation repository**
  - https://readthedocs.org
- **And Linux, of course!**



We see further when 'we stand on the shoulders of giants'

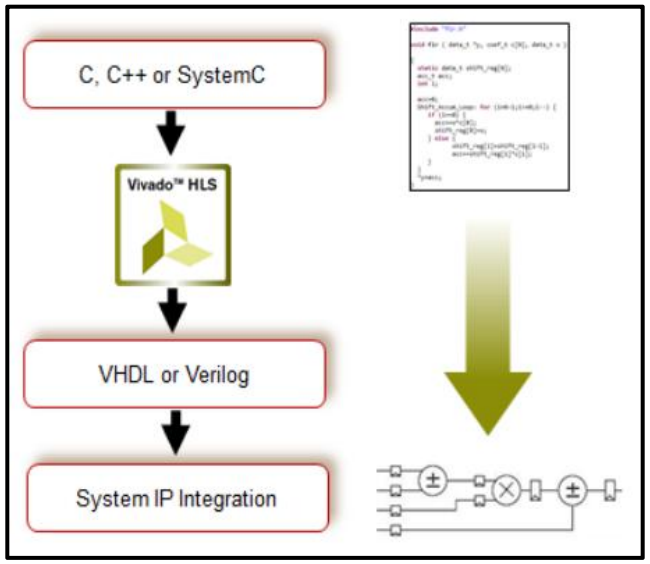
Python Productivity on Zynq

Today Zynq uses systems/efficiency languages

Could Zynq flows exploit productivity languages?

Productivity languages

Application oriented



Cross-compiled C/C++ code

FreeRTOS/Linux

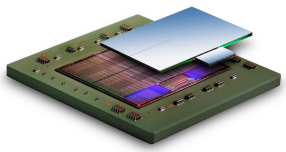
Bare metal IP drivers in C

Zynq PS & PL Hardware

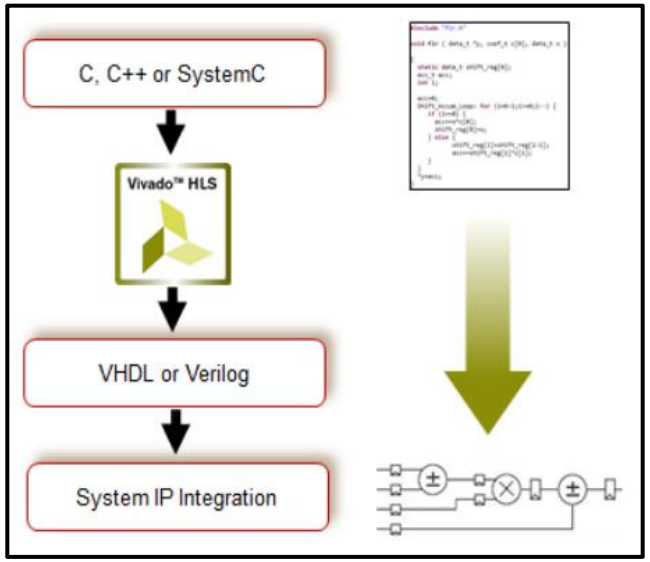
Systems/ efficiency languages

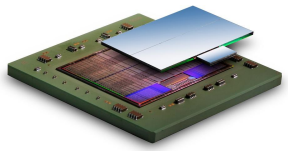
H/W flow

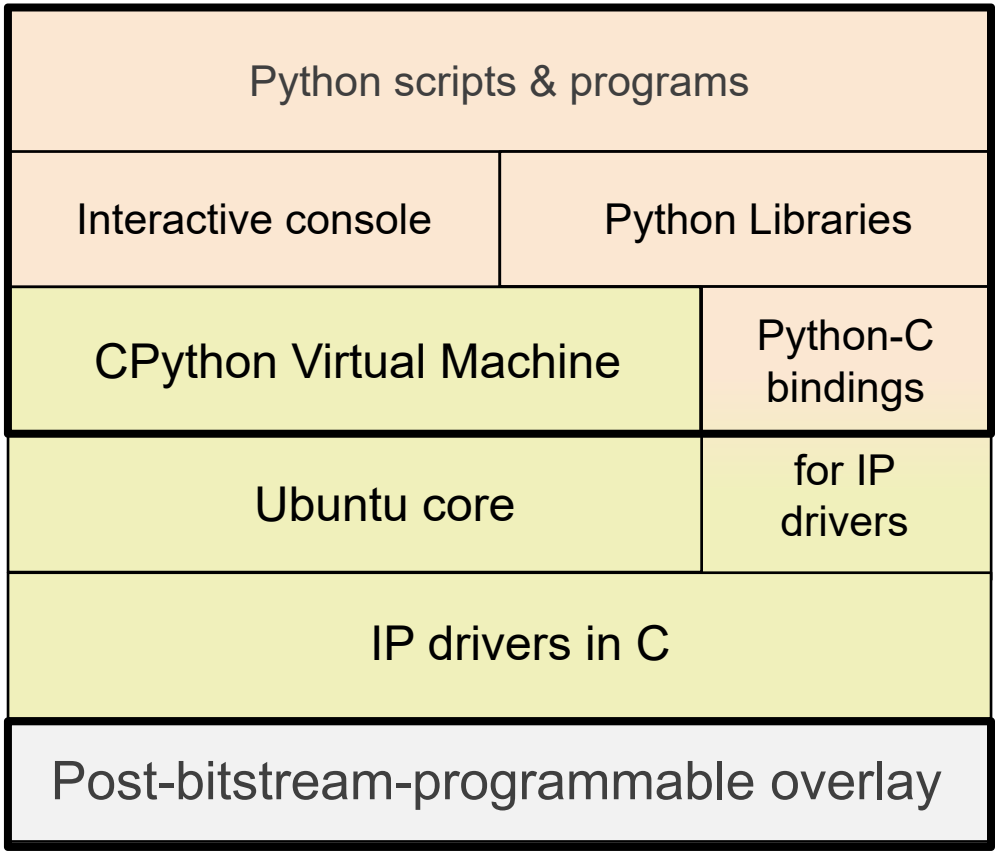
S/W flow



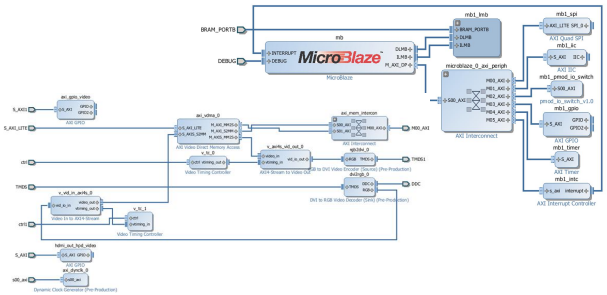
Productivity level tools for Zynq







Overlays with IOP



Step 1:

Create an FPGA design for a class of related applications

```
pmod_init(0,1);
while(1){
    while((MAILBOX_CMD_ADDR & 0x01)
    cmd=MAILBOX_CMD_ADDR;

    count = (cmd & 0x0000ff00) >>
    if((count==0) || (count>253)) {
        // clear bit[0] to indicate
        // set rest to 1s to indica
        MAILBOX_CMD_ADDR = 0xffffff
        return -1;
    }
    for(i=0; i<count; i++) {
        if (cmd & 0x08) // Python
        {
            switch ((cmd & 0x06) >> 1) { // use bit[2:1]
                case 0 : MAILBOX_DATA(i) = *(u8 *) MAILBOX_ADDR; break;
                case 1 : MAILBOX_DATA(i) = *(u16 *) MAILBOX_ADDR; break;
                case 2 : break;
                case 3 : MAILBOX_DATA(i) = *(u32 *) MAILBOX_ADDR; break;
            }
```

```
6c78 3963 7367 3232 3500 6300 0b32
332f 3039 2f33 3000 6400 0931 323a
3a31 3900 6500 0532 7cff ffff ffff
ffff ffff ffff ffff ffaa 9955 6630
0720 0031 a103 8031 413d 0831 6109
c204 0010 9330 e100 cf30 c100 8120
0020 0020 0020 0020 0020 0020 0020
0020 0020 0020 0020 0020 0020 0020
813c c831 8108 8134 2100 0032 0100
e1ff ff33 2100 0533 4100 0433 0101
6100 0032 8100 0032 a100 0032 c100
```

Step 2:

Export the bitstream and a C API for programming the design

```
void setNormalDisplay(){
    sendCommand(OLED_Normal_Display_Cmd);
}

void setInverseDisplay(){
    sendCommand(OLED_Inverse_Display_Cmd);
}

int main(void)
{
   int cmd;
   int Row, Column;

   arduino_init(0,0,0,0);
   config_arduino_switch(A_GPIO, A_GPIO, A_GPIO,
                         A_GPIO, A_SDA, A_SCL,
                         D_GPIO, D_GPIO, D_GPIO, D_GPIO, D_GPIO,
                         D_GPIO, D_GPIO, D_GPIO, D_GPIO,
                         D_GPIO, D_GPIO, D_GPIO, D_GPIO);
   // Initialization
   oled_init();
```

Step 3:

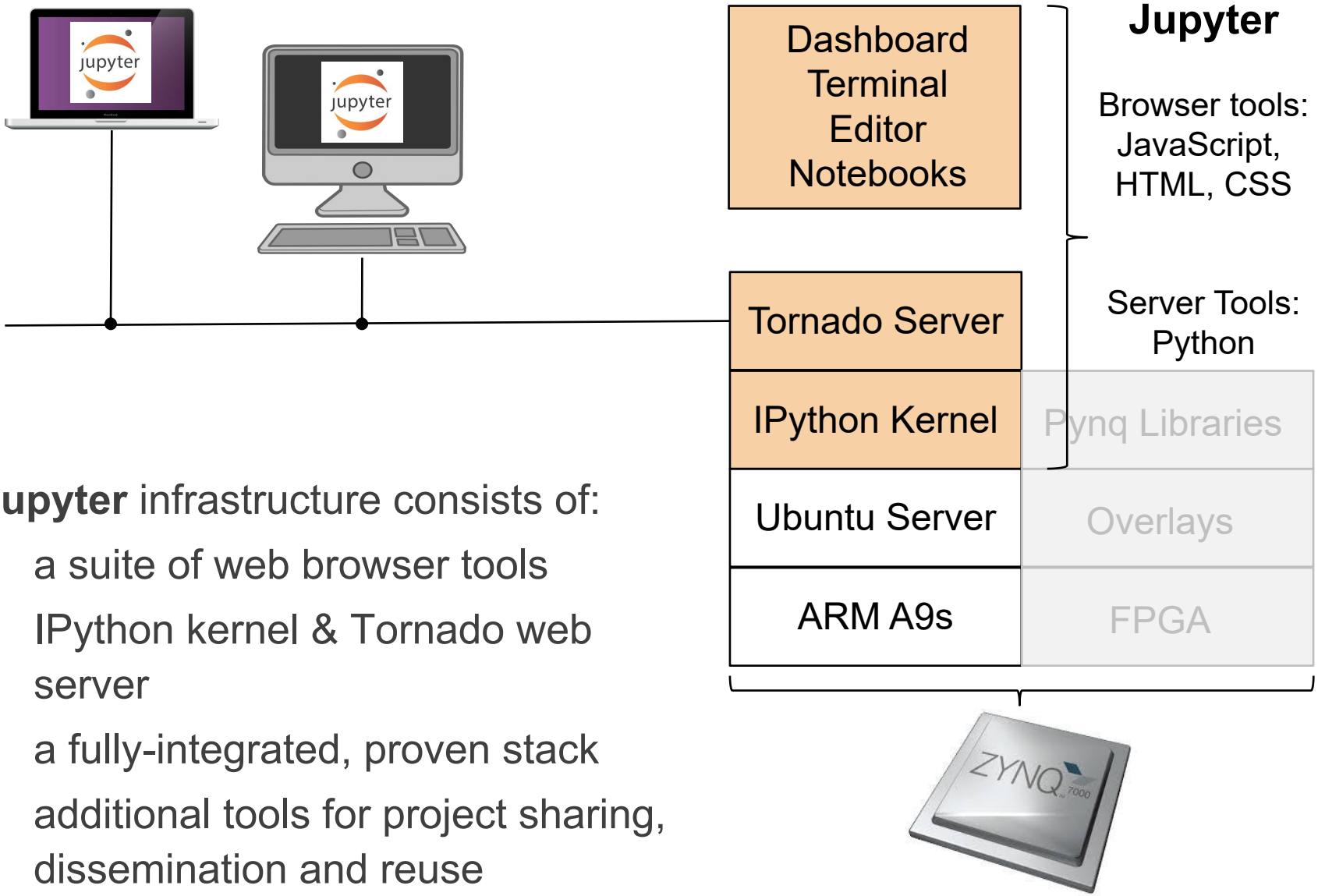
Wrap the C API to create a Python library

```
from time import sleep
from pynq import Overlay
from pynq.iop import PMOD_ADC, PMOD_DAC

ol = Overlay("base.bit")
ol.download()
# Writing values from 0.0V to 2.0V with step 0.1V.
dac_id = int(input("Type in the PMOD ID of the DAC (1 ~ 2): "))
adc_id = int(input("Type in the PMOD ID of the ADC (1 ~ 2): "))

dac = PMOD_DAC(dac_id)
adc = PMOD_ADC(adc_id)

for j in range(20):
    value = 0.1 * j
    dac.write(value)
    sleep(0.5)
#    readings=adc.read(1,0,0)
#    x1=readings[0]
    print("Voltage read by DAC is: {:.4f} Volts".format(adc.read(1,0,0)[0]))
```

Step 4:

Import the bitstream and the library in your Python scripts and program

Jupyter on Zynq



Jupyter infrastructure consists of:

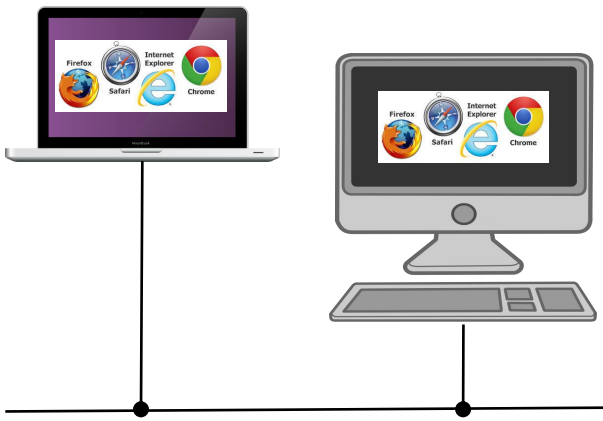
- a suite of web browser tools
- IPython kernel & Tornado web server
- a fully-integrated, proven stack
- additional tools for project sharing, dissemination and reuse

FPGA overlays – hardware libraries (kernels)

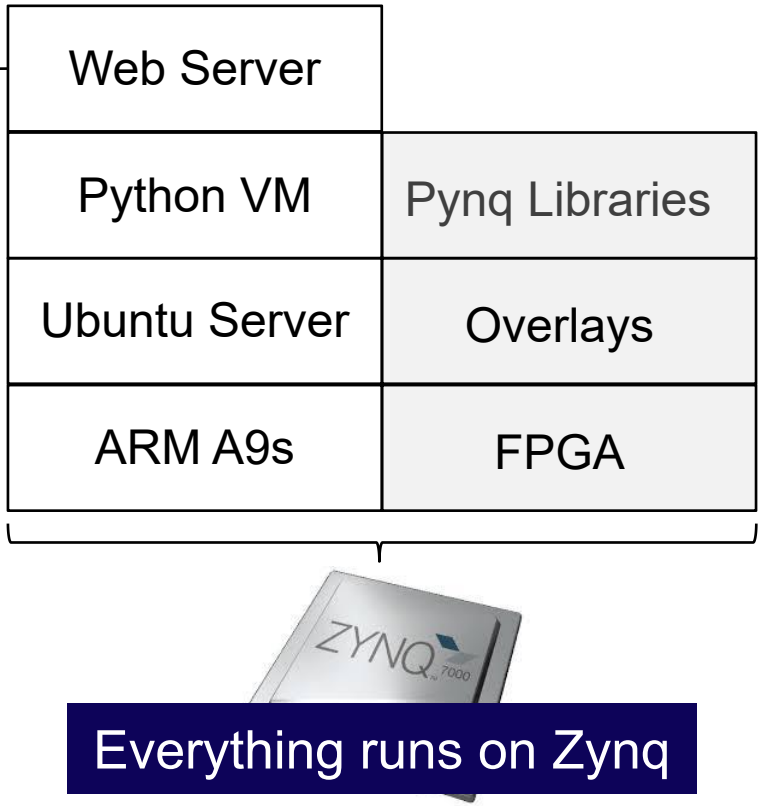
- **Overlays are generic FPGA designs that target multiple users with new design abstractions and tools**
  - Post-bitstream programmable via software APIs
  - Typically optimized for given domains
  - Encourages the use of open source tools & fast compilation
  - Enables productivity from re-using pre-optimized designs
  - Exposes benefits of FPGAs to new users

- **Active research area with many papers**

Web & Python-based, open source architecture



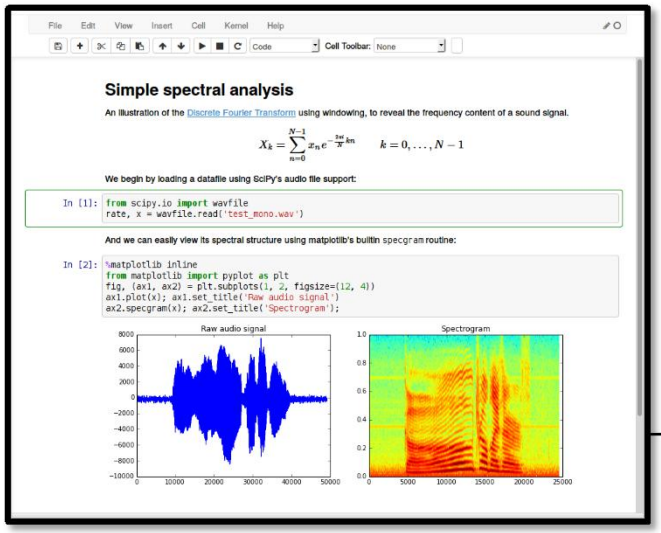
Develop on the target, access via the browser

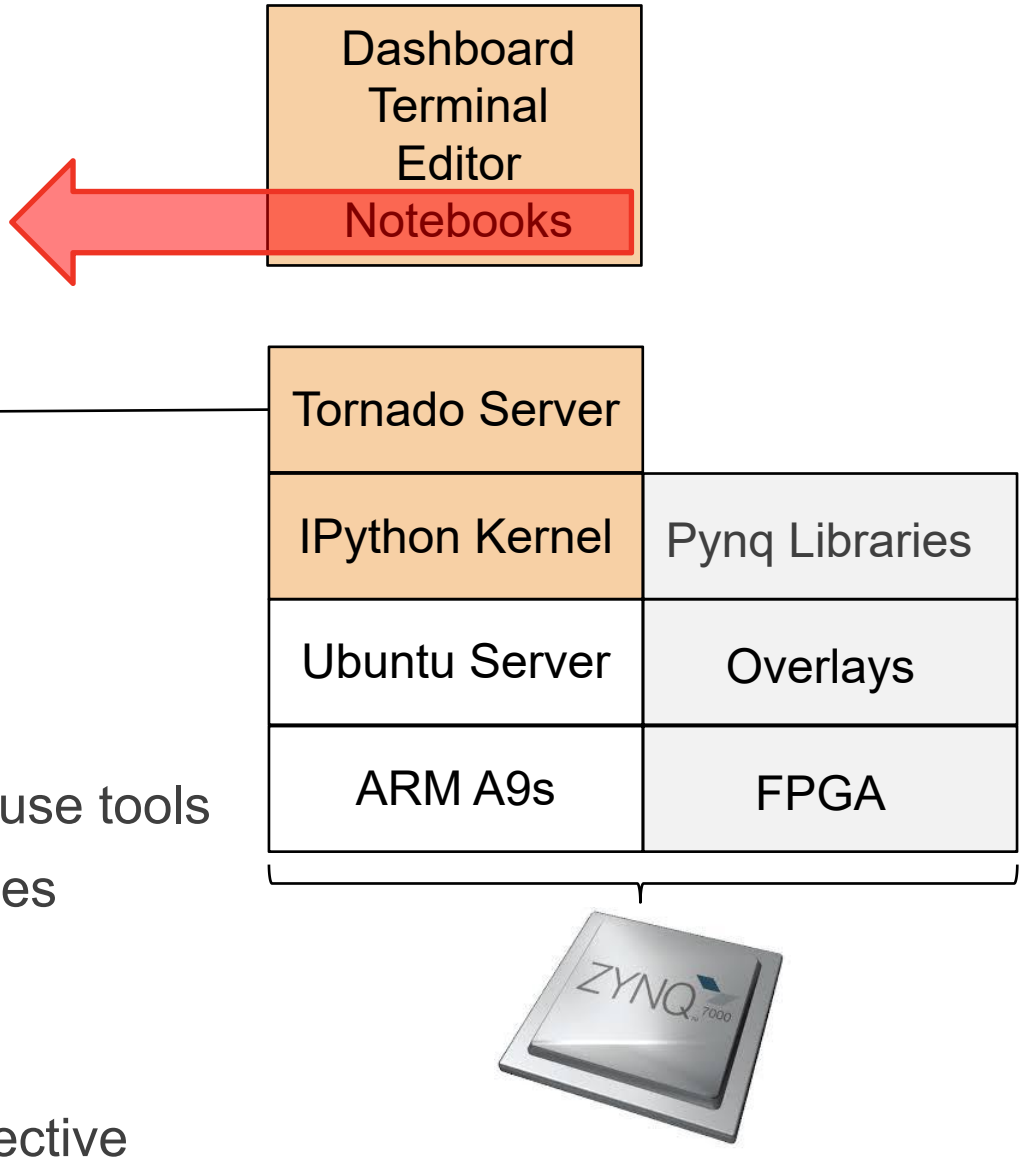


Architecture emphasizes :

- a software-centric approach
- based on open, de facto standards
- platform, OS and browser agnostic
- minimal learning curve
- no proprietary methodologies

Jupyter on Zynq





Jupyter Notebooks

Interactive design with Zynq:

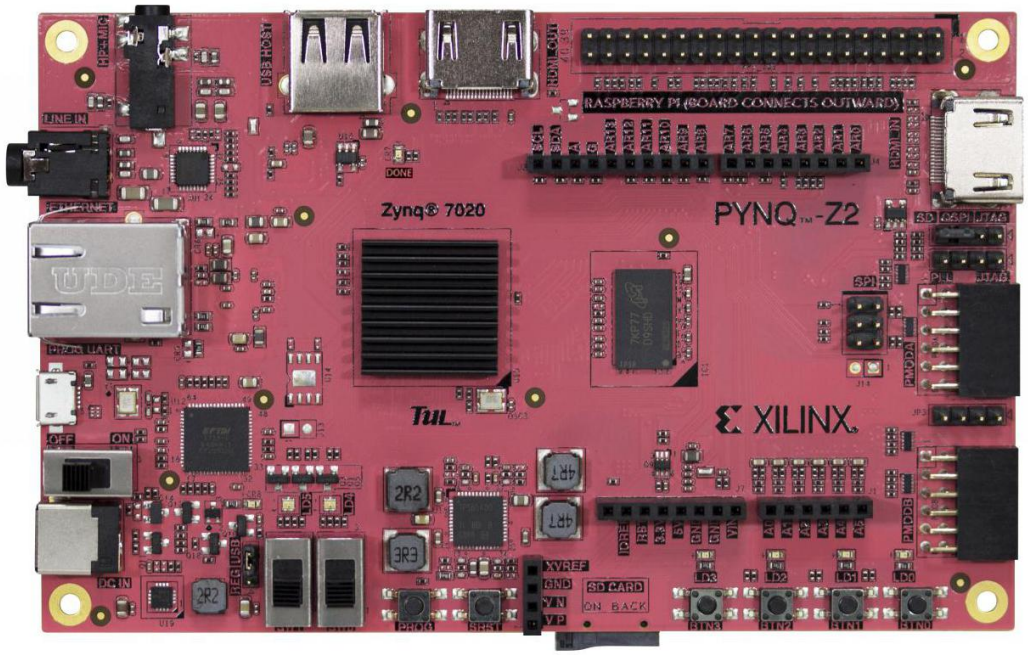
- highly productive, easy-to-use tools
- extensive third-party libraries
- integrated multi-media
- open source JSON format
- application-oriented perspective

With **PYNQ** embedded programmers **get** …

- **Productivity-level programming**
  - Interactive console & scripting
  - Fast compile and debug times
- **Hardware performance in a software environment**
  - Hardware parallelism
  - Function independence
  - Predictable performance
  - Hard, real-time latencies
- **Portable libraries & open source community**

PYNQ™-Z2 Platform

Low-cost PYNQ-Z2 Board



- Zynq XC7Z020 CLG400
- 512 MB DRAM
- HDMI input, HDMI out
- Microphone input
- Audio output
- 10/100/1G Ethernet
- MicroSD slot
- USB 2.0
- 4 LEDs, 4 buttons, 2 switches
- Arduino connector
  - With additional IO
- 2 Pmod 8-pin GPIOs

Available from E-ELEMENTS for approved academic customers

Wide range of low-cost sensors, actuators, etc

Environmental Monitoring

Have you ever wanted to get your daily weather report based on data from your garden instead of obtaining a more generic report from your TV or mobile phone? Sensors


Grove - Digital Light Sensor


Grove - Light Sensor


Grove - Temperature and Humidity Sensor


Grove - Barometer Sensor


Grove - Dust Sensor

Motion Sensing

Sensors in this category enable your microcontroller to detect motion, location and direction. You can make the movement of your microcontroller understandable in three dimensional spaces


Grove - 3-Axis Digital Compass


Grove - 3-Axis Digital Accelerometer(±1.5g)


Grove - 3-Axis Digital Gyro


Grove - Collision Sensor


Grove - 3-Axis Analog Accelerometer

Wireless Communication

Communicating without wires is a cool feature that can spice up your project. Modules in this category arm your microcontroller with wireless communication ability such as RF, Bluetooth, etc.


Grove - 315MHz Simple RF Link Kit


Grove - Serial RF Pro


Grove - GPS


Grove - 125KHz RFID Reader


Grove - Serial Bluetooth

User Interface

Modules in this, our largest, category, let you interface with your microcontroller via input modules, such as touch pads, joysticks or your voice. Or you can choose output modules,


Grove - Solid State Relay


Grove - OLED Display 128*64


Grove - Serial LCD


Grove - LED Socket Kit


Grove - Button

Physical Monitoring

Scientists understand the world around us in physical dimensions. Modules in this category are designed to help you analyze the physical world. Measure your heart rate,


Grove - Water Sensor


Grove - Magnetic Switch

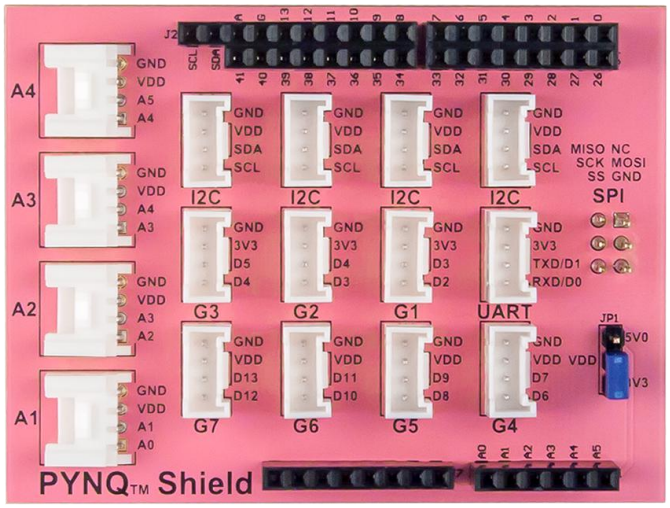

Grove - Alcohol Sensor


Grove - RTC


Grove - Differential Amplifier

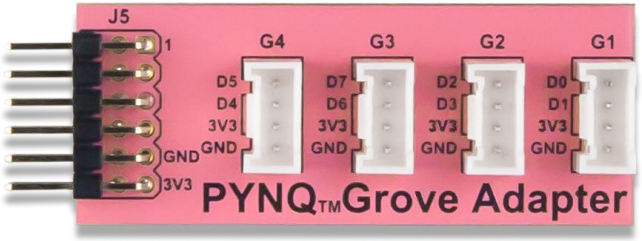
www.seeedstudio.com/wiki/Grove_System

Low-cost Pynq Grove Shield & Pmod Adapter



Grove Shield:

- 4 x Analog ports
- 4 x I2C ports
- 3 x 3.3V GPIO ports
- 1 x UART
- 4 x 3.3/5V switchable GPIO ports
- 1 x SPI header
- 1 x 16-pin GPIO header

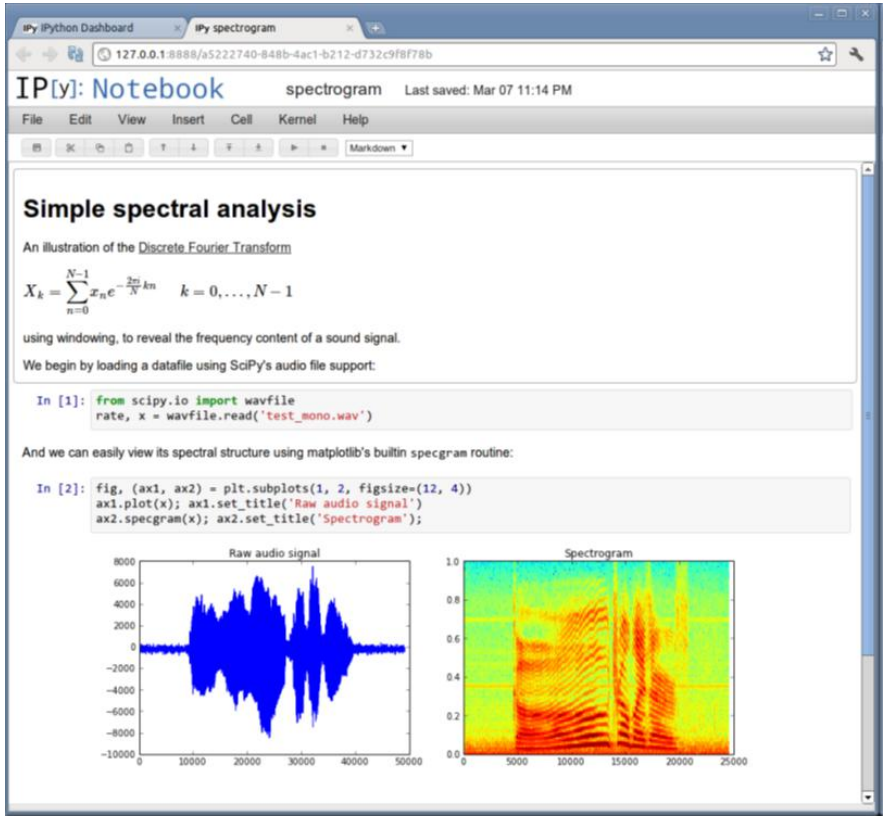


Pmod2Grove Adapter :

- 4 independent sockets for Grove modules
- Pmod compatible
- Solderless breadboard compatible
- Open-source design

Jupyter Notebook

Jupyter Notebooks …
a new paradigm for reproducible knowledge

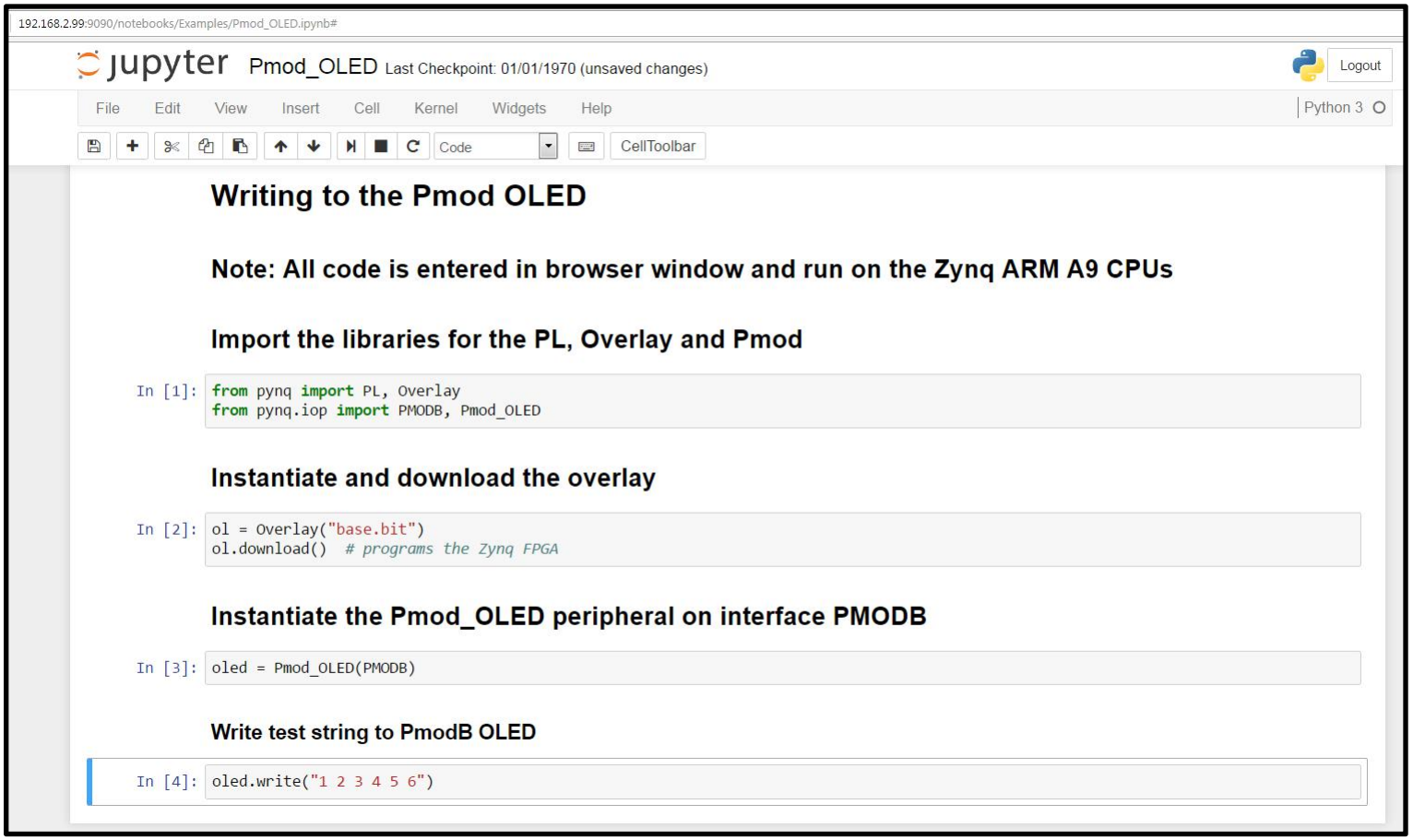


- Designed for
  - Interactive, exploratory computing
  - Reproducible results
- Ideal for
  - Teaching and learning
  - Projects and research
- Rapid adoption
  - Over 500,00 notebooks uploaded to Github in the last 3 years

github.com/ipython/ipython/wiki/A-gallery-of-interesting-IPython-Notebooks

Coming shortly for Zynq APSoCs as part of the PYNQ open source project

Jupyter Notebook



Jupyter Notebook Code Cells

```
In [1]: from pynq import PL, Overlay
        from pynq.iop import PMODB, Pmod_OLED

In [2]: ol = Overlay("base.bit")
        ol.download()  # programs the Zynq FPGA

In [3]: oled = Pmod_OLED(PMODB)

In [4]: oled.write("1 2 3 4 5 6")
```

6 lines of user code … thanks to Python, FPGA overlays, abstraction & re-use